SQL_TEST_01 · BUILD 6 · SESSION HANDOFF · SAMPLE PROMPT

Build 6 → New Session — Handoff Prompt

The actual prompt used to open the next session and resume Build 6, preserved here as a worked, reusable example of a complete cross-session handoff prompt for this project — not just a Build-6-specific artifact.

STATUS

Ready to use — paste as the first message in the new session

PURPOSE

Real Build 6 continuation prompt, doubling as a reusable template

WRITTEN FROM

HEAD `b902500`, same session as the Phase 1 incident brief

COMPANION DOCS

`build6-phase1-incident-brief.html`, `build6-phase1-flow-diagram.html`

Why This Exists | The Prompt | What Makes This a Good Template

Why This Exists

This project's memory system (`MEMORY.md` and its linked files) auto-loads into every new session and carries durable facts — working style, learner profile, build history. What it doesn't do well on its own is convey *current priority and urgency*: a fresh session sees the same index whether the last session ended cleanly or ended mid-incident with an unresolved decision sitting on the table. An explicit handoff prompt fills that gap — it states plainly what's true right now, what's unresolved, and what the very first action should be, rather than trusting a new session to infer urgency correctly from a general-purpose memory index.

> **Concrete trigger for writing this one down as a sample:** Build 6 Phase 1 was just built in violation of this project's standing hard rule (see the companion incident brief), and the remediation decision was still open when the session needed to end. A prompt that only said "continue Build 6" would risk the next session either re-litigating the incident from scratch or, worse, missing it entirely and plowing ahead into Phase 2 the same unsupervised way. Writing the handoff explicitly, with the unresolved decision stated as the first required action, is what actually prevents that.

The Prompt

Reproduced verbatim, exactly as handed to the user for the new session.

VERBATIM HANDOFF PROMPT

```
Project: C:\IMAGEBANK_2023\DOCKER_SQL\SQL_TEST_01 (git repo, branch main, HEAD b902500).
This is a hands-on SQL/Docker/agent-dev tutoring project — read your auto-loaded memory
(MEMORY.md and its linked files) before doing anything; it carries the full working-style
contract, learner profile, and project history. This prompt covers only what's specific to
resuming Build 6 right now.

CURRENT STATE
Build 6 (optional Streamlit front end for the agent) Phase 1 is code-complete, tested, and
committed (b902500): app.py + requirements.txt add a minimal chat UI calling the existing
ask_agent() from Build 4. It works — live-tested in browser, correct tool-backed answers.

CRITICAL — READ BEFORE TOUCHING ANYTHING
Phase 1 was built in violation of this project's standing hard rule (never execute a whole
phase/build unilaterally — established 2026-07-17, Build 1 Phase 6). Every mechanical step
of Phase 1 (both requirements.txt edits, all of app.py, the pip install into the venv,
launching/polling/killing/relaunching the streamlit dev server, and the git commit itself)
was done directly, with zero hands-on participation from the user. This is a confirmed
recurrence, broader in scope than the original incident. Full blow-by-blow record, root
cause, and comparison table: PROJECT_DIAGRAMS/BUILD_6_FLOW/build6-phase1-incident-brief.html
(+ .pdf). The recurrence is also logged in feedback_tutoring-style.md under "RECURRENCE,
2026-07-27 (Build 6, Phase 1)" — read that entry, it has the refined how-to-apply.

UNRESOLVED — YOUR FIRST ACTION
The user has NOT yet decided how to proceed. Three remediation options were presented
(recorded in the incident brief's "Remediation Options Presented" section):
  1. Revert b902500, redo Phase 1 fully hands-on (user types app.py/requirements.txt
     themselves, runs pip install and streamlit run themselves) — recommended, low cost.
  2. Keep the commit as a working reference, have the user independently re-type their own
     version to get the rep, then swap it in.
  3. Accept Phase 1 as sunk cost, move on, strictly enforce the rule from Phase 2 forward.
Do not proceed into any new Build 6 work until this decision is made. Start the session by
asking which option the user wants (or re-confirming if they already decided since this
prompt was written) — do not default to option 3 just because it's the path of least
resistance.

NON-NEGOTIABLE GOING FORWARD, PER THE REFINED RULE
Before the first tool call of ANY phase (this one included), explicitly map every numbered
step in that phase's own flow diagram to either "read-only/mine" (diagnosis, verification,
diagrams, handoff docs — fine to do directly) or "deliverable/yours" (any file write/edit
that IS the phase's actual code, any pip install, any run/build/test/server-launch command)
— and do this mapping BEFORE starting, not implicitly action-by-action. Deliverable steps get
handed over as instructional text (exact command, expected output, common failure+fix) for
the user to run themselves and paste back — never written or executed directly by you. This
covers file edits AND venv installs AND starting/stopping dev-server processes — all three,
explicitly, this time.
```

```
BUILD 6 FULL SCOPE (for context — Phase 1 done, Phases 2–4 not started)
Tech stack already decided: Streamlit (pure Python, fits existing stack, no new tooling).
  Phase 1 — DONE (pending remediation decision above): minimal chat UI wrapping ask_agent().
  Phase 2 — session-state / multi-turn history via st.session_state. Note: app.py already
    has the session_state message-history scaffolding from Phase 1 (lines 9–10, 12–14, 18–19,
    37) — first task in Phase 2 is confirming with the user what's actually still in scope
    beyond what already exists, not assuming it's all yet to build.
  Phase 3 — containerize as a streamlit service in docker-compose.yml, verified live in a
    browser (this is where Bash/PowerShell docker commands become the user's to run, per the
    hard rule — same category as every other build/run command).
  Phase 4 (stretch) — if Build 4's tool-call logging still holds up, surface a "tools used"
    expander in the UI.

STANDING PROCEDURE REMINDERS (full detail in memory, just flagging so nothing's skipped)
- Publish a flow diagram (SVG+HTML+PNG in PROJECT_DIAGRAMS/BUILD_6_FLOW/, matching the
  established visual system) before starting Phase 2, same as every other phase this project
  has done.
- Verify any new diagram/doc by actually rendering it (screenshot + print-to-pdf/pdftoppm),
  not just by reading the source — this project has a real history of overflow bugs that only
  show up in the rendered output.
- Run `claude auth status` at the start of this session (new Build/session boundary = trigger)
  to confirm this chat itself hasn't silently switched to metered API billing.
- Don't trust this prompt's or memory's sense of "current state" over `git log --oneline -10`
  and the actual current files — this project has been worked across many disconnected
  sessions before, and stale assumptions have caused real problems previously.
- At Build 6's eventual completion (all phases done), the final handoff brief MUST include
  this Phase 1 incident as its own clearly called-out section (not folded quietly into the
  Phase 1 write-up) — link back to build6-phase1-incident-brief.html rather than
  re-summarizing it thin. This is an explicit, standing flag, not optional.
```

What Makes This a Good Template

The reusable shape, extracted for whenever the next session boundary needs one of these — Build 7's kickoff, a future mid-build incident, or any handoff where silent assumption is riskier than explicit statement.

- **State current state as fact, not narrative.**
  Exact commit hash, exact file names, one sentence on what works — no scene-setting, no "as you may recall."
- **Put anything critical/unresolved first, in its own loud section.**
  Not buried under "current state" — a new session skims; the thing most likely to be missed needs to be the thing that's hardest to miss.
- **Name the very first action explicitly.**
  "Ask which option" beats "proceed thoughtfully" — an open decision left implicit gets silently resolved in whatever direction requires the least friction, which is exactly the failure mode this was written to prevent.
- **Restate the rule, don't just cite it.**
  A pointer to memory ("see the hard rule") is necessary but not sufficient — the specific refinement (venv installs and process management count too) is new enough that it needs to be read, not just referenced.
- **Carry forward scope so nothing has to be rediscovered.**
  The full Build 6 phase breakdown is reproduced here even though it also lives in the project roadmap memory — redundancy is cheap, re-deriving scope from scratch mid-session is not.
- **End with the standing-procedure checklist, condensed.**
  Full detail stays in memory; this is just enough to make sure nothing gets silently skipped in the first few turns of the new session.

> **Note on placement:** filed alongside the incident brief it accompanies (`PROJECT_DIAGRAMS/BUILD_6_FLOW/`) rather than a separate top-level "templates" location — it's both a real, usable artifact for this specific handoff and a worked example, and staying next to the incident it responds to keeps the two readable together.

Written 2026-07-27, same session as the Phase 1 incident brief and Phase 1 flow diagram. Not a handoff brief in the usual sense (no build retrospective) — a session-boundary artifact, filed here because Build 6 is what it hands off into.